[Dat Nguyen] IPC: Pipes, FIFOs, Message Queues, Shared Memory, Signals

IPC (Inter-Process Communication)

Inter-Process Communication (IPC) is a set of mechanisms provided by the operating system. It allows independent processes to exchange data, share information, and synchronize their activities.

Purpose	Meaning	Example
Exchange data	One process sends data to another process.	Process A sends a message to Process B.
Share information	Multiple processes can access the same information or data.	Two processes use the same shared memory.
Synchronize activities	Processes coordinate their actions so they work in the correct order.	Process B waits until Process A finishes writing data.

1. Pipes, popen/pclose Functions, Coprocesses, FIFOs

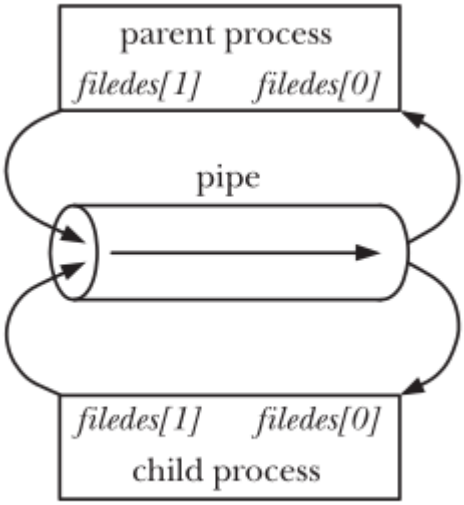
1.1 Pipe

A pipe creates a one-way data channel between processes.

Scope: It can only be used between related processes, such as a parent and child process, which are created using `fork()`.

create pipe : `int pipefd[2]` => The kernel returns two file descriptors:

- pipefd[0] → Read end: Used to read data from the pipe.
- pipefd[1] → Write end: Used to write data to the pipe.



a) After *fork()*

Pipe Capacity: The default pipe capacity on this system is 65,536 bytes.

Example:

```
int main()
{
    int fd[2];

    pipe(fd);

    int size = fcntl(fd[0], F_GETPIPE_SZ);

    printf("Pipe capacity: %d bytes\n", size);

    close(fd[0]);
    close(fd[1]);

    return 0;
}
```

Output:

```
Pipe capacity: 65536 bytes
```

The maximum pipe size allowed by the system : 1048576

Setting up a pipe to transfer data from a parent to a child

- After `fork()`, both the parent and child process have access to both ends of the pipe.

- Technically, both processes can read from and write to the pipe. However, this is rarely used in practice because it can cause problems. For example, a process may read the data that it just wrote.
- If both processes try to read from the same pipe at the same time, we cannot know which process will read the data first. This can cause a race condition.
- To create a clear one-way data flow, each process should close the file descriptor that it does not use immediately after `fork()`.

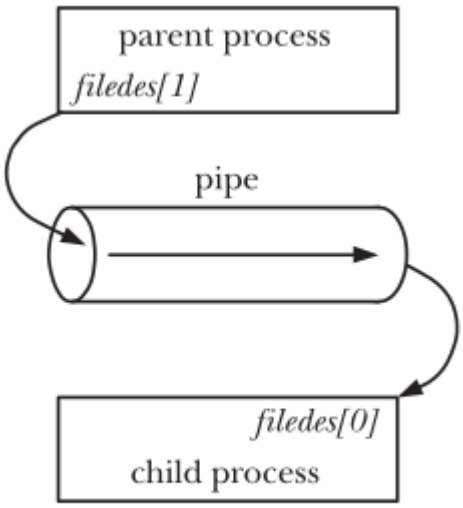
For example: Parent → Child:

Parent:

- Close the read end.
- Write data to the pipe.

Child:

- Close the write end.
- Read data from the pipe.



b) After closing unused descriptors

Code

```
int main() {
   int filedes[2];
   pipe(filedes);

   pid_t pid = fork();

   if (pid > 0) {
       // --- PARENT PROCESS ---
```

```
        close(filedes[0]); // Parent closes read end



        char message[] = "Hello!";
        write(filedes[1], message, strlen(message) + 1);

        close(filedes[1]); // Parent finishes writing and CLOSES write end
    }
    else if (pid == 0) {
        // --- CHILD PROCESS ---

        close(filedes[1]); // Close unused write end

        char buffer[100];

        // Read 1: Successfully read the "Hello!" string from parent
        read(filedes[0], buffer, sizeof(buffer));
        printf("Read 1 received: %s\n", buffer);

        // Read 2: Try to read more until End-Of-File (EOF)
        printf("Waiting to receive more...\n");
        ssize_t bytes = read(filedes[0], buffer, sizeof(buffer));

        // THIS PART WILL RUN NOW AFTER CLOSING filedes[1]!
        printf("Bytes read: %ld\n", bytes);
        close(filedes[0]);
    }

    return 0;
}
```

Output:

```
Read 1 received: Hello!
Waiting to receive more...
Bytes read: 0
```

1.2 popen/pclose Functions

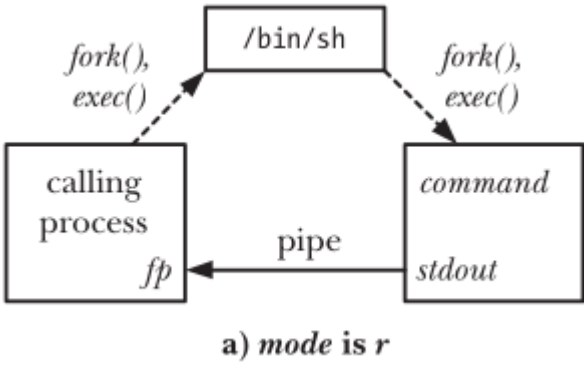
A common use for pipes is to execute a shell command and either read its output or send it some input

popen: used to read the `stdout` of a shell command or write to its `stdin` directly from a C program through a `FILE*` stream.

pclose() : used to close the pipe (stream) that was opened by `popen()`. It also waits for the child process to finish and returns the exit status of the command.

- `"r"`: The parent process reads the output from the child process.
- `"w"`: The parent process writes data to the input of the child process.

Mode r:



a) *mode* is *r*

Code:

```
int main() {
   char buffer[256];

   // 1. The calling process creates a pipe, then calls fork/exec to run /bin/sh -c "ls -l ;
pwd""
   FILE *fp = popen("ls -l; pwd"", "r");
   if (fp == NULL) {
       perror("popen failed");
       return 1;
   }

   // 2. Read data from the command's stdout through the pipe
   while (fgets(buffer, sizeof(buffer), fp) != NULL) {
       printf("Read from pipe: %s", buffer);
   }

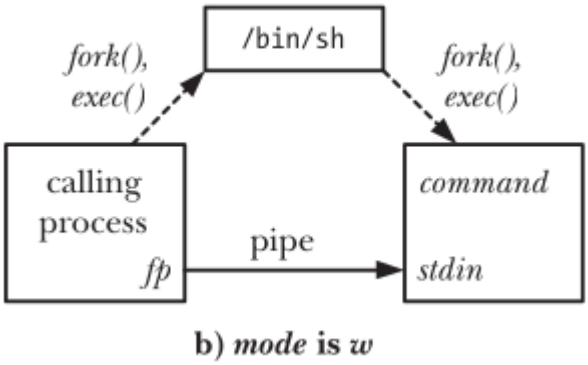
   // Close the pipe and clean up the process
   pclose(fp);
```

```
    return 0;
}
```

Output:

```
Read from pipe: -rwxrwxr-x 1 dat dat 16184 Sep 15 21:06 app
Read from pipe: -rw-rw-r-- 1 dat dat  2735 Sep 15 13:44 reader.c
Read from pipe: -rw-rw-r-- 1 dat dat  6668 Sep 15 15:08 writer.c
Read from pipe: /home/dat/Documents/Zhone-Training/Section4
```

Mode w:



b) *mode* is *w*

Code:

```
int main() {
    // 1. calling process create pipe, fork/exec run /bin/sh -c "grep hello | wc -l"
    // Open pipe in mode w "w" (write)
    FILE *fp = popen("grep hello | wc -l", "w");
    if (fp == NULL) {
        perror("popen failed");
        return 1;
    }

    // 2.Write data from Parent into pipe
    fprintf(fp, "line 1: hello world\n");
    fprintf(fp, "line 2: foo bar\n");
    fprintf(fp, "line 3: say hello again\n");
    sleep(2);

    // Close pipe
    pclose(fp);
```

```
    return 0;
}
```

Output:

```
2
```

Explain

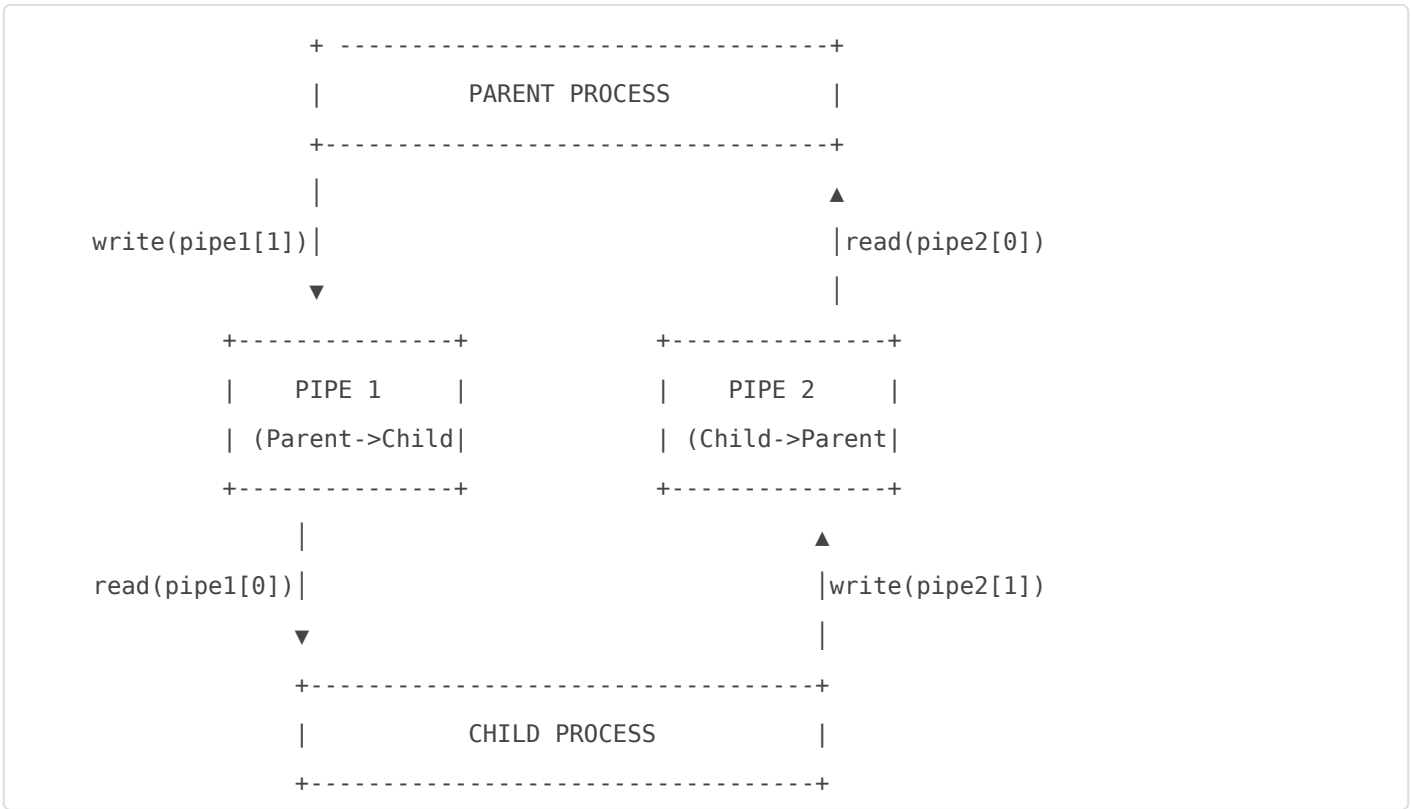
- grep hello: Filters out the 2 matching lines (line 1 and line 3) and sends the output to wc -l via the pipe.
- wc -l: Counts the number of lines received from grep and prints the total count.

Note: /bin/sh

- a program file located in the `/bin` directory. `sh` stands for **shell**.
- It is a command-line interpreter. It does not directly perform tasks such as listing files or filtering text. Instead, it reads commands, understands their syntax, and then finds and runs the appropriate programs.

1.3 Coprocesses

Setting up 2 pipe to transfer data from a parent to a child and a child to a parent



Code

```
int main() {
   int parent_to_child[2]; // Pipe 1: Parent -> Child
   int child_to_parent[2]; // Pipe 2: Child -> Parent

   // 1. Initialize both pipes
   if (pipe(parent_to_child) < 0 || pipe(child_to_parent) < 0) {
       perror("Pipe creation failed");
       exit(1);
   }

   pid_t pid = fork();

   if (pid > 0) {
       // ================= PARENT PROCESS =================
       // Close unused pipe ends (Cross-closing)
       close(parent_to_child[0]); // Parent does NOT READ from Pipe 1
       close(child_to_parent[1]); // Parent does NOT WRITE to Pipe 2

       char send_msg[] = "hello coprocess";
       char recv_msg[100];

       // Send data to the child
       printf("[PARENT] Sending: %s\n", send_msg);
       write(parent_to_child[1], send_msg, strlen(send_msg) + 1);

       // Read the response back from the child
       read(child_to_parent[0], recv_msg, sizeof(recv_msg));
       printf("[PARENT] Received back: %s\n", recv_msg);

       // Close remaining pipe ends after use
       close(parent_to_child[1]);
       close(child_to_parent[0]);

   } else if (pid == 0) {
       // ================= CHILD PROCESS =================
       // Close unused pipe ends (Cross-closing)
       close(parent_to_child[1]); // Child does NOT WRITE to Pipe 1
       close(child_to_parent[0]); // Child does NOT READ from Pipe 2

       char buffer[100];
```

```
        // Read data sent by the parent
        read(parent_to_child[0], buffer, sizeof(buffer));

        // Process data: Convert to uppercase
        for (int i = 0; buffer[i] != '\0'; i++) {
            buffer[i] = toupper((unsigned char)buffer[i]);
        }

        // Send the modified string back to the parent
        write(child_to_parent[1], buffer, strlen(buffer) + 1);

        // Close remaining pipe ends
        close(parent_to_child[0]);
        close(child_to_parent[1]);

        exit(0);
    }


    return 0;
}
```

Output

```
[PARENT] Sending: hello coprocess
[PARENT] Received back: HELLO COPROCESS
```

Explain

```
Pipe 1 (p1): Parent Writes -> Child Reads.

Pipe 2 (p2): Child Writes -> Parent Reads.
```

- Parent Process

Task: Writes to Pipe 1 (p1[1]) and Reads from Pipe 2 (p2[0]).

```
Unused FDs -> Must close() immediately:
```

```
    p1[0] (Pipe 1 Read end – Parent does not read Pipe 1).

    p2[1] (Pipe 2 Write end – Parent does not write to Pipe 2).

Action in Parent: close(p1[0]); and close(p2[1]);
```

- Child Process

Task: Reads from Pipe 1 (p1[0]) and Writes to Pipe 2 (p2[1]).

```
Unused FDs -> Must close() immediately:

    p1[1] (Pipe 1 Write end – Child does not write to Pipe 1).

    p2[0] (Pipe 2 Read end – Child does not read Pipe 2).

Action in Child: close(p1[1]); and close(p2[0]);
```

1.4 FIFO

FIFO = First In, First Out.

FIFO is similar to a pipe. The principal difference is that a FIFO has a name within the file system and is opened in the same way as a regular file. This allows a FIFO to be used for communication between unrelated processes

Writer

```
int main() {
   mkfifo("/tmp/myfifo", 0666);

   // Block until process B open O_RDONLY
   int fd = open("/tmp/myfifo", O_WRONLY);

   write(fd, "Hello", 5);
   close(fd);
   return 0;
}
```

Reader:

```
int main() {
   // Wake up process A is blocking
   int fd = open("/tmp/myfifo", O_RDONLY);

   char buffer[100];
   int n = read(fd, buffer, sizeof(buffer) - 1);
   if (n > 0) {
       buffer[n] = '\0';
       printf("Recieved: %s\n", buffer);
   }

   close(fd);
   return 0;
}
```

Ouput

```
Recieved: Hello
```

Explain

- FIFO data is not stored on disk but in temporary Kernel RAM (usually around 64KB).
- The writer will block at open() until the reader starts.
- Thanks to the blocking mechanism, the Kernel puts the writer into a sleep state. The writer uses 0% CPU until the reader appears and wakes it up.
- It doesn't matter which process starts first. Whichever runs first will wait at open(). When both are present, the connection is established.

Case: O_NONBLOCK Flag

- int fd = open("/tmp/myfifo", O_WRONLY | O_NONBLOCK);
- If no reader exists: open() fails immediately, returns -1, and sets errno = ENXIO.
- If a reader exists: Opens successfully as normal.

2. IPC Identifiers and Keys, Permission Structure, Configuration Limits, Advantages/Disadvantages

2.1 IPC Identifiers and Keys

Key: A value used to identify the IPC object a process wants to create or access.

IPC ID: An identifier used by the process to interact with the object after it is found.

- **Generating a unique key with IPC_PRIVATE**

```
int msgid = msgget(IPC_PRIVATE, 0666);
```

This technique is especially useful in multiprocess applications where the parent process creates the IPC object prior to performing a fork(), with the result that the child inherits the identifier of the IPC object

- **Generating a unique key with ftok()**

```
key_t ftok(const char *pathname, int proj_id);
```

Example: key_t key = ftok("/tmp/myfile", 'A');

Note:

```
ftok(): Converts a file path into an IPC key.

'A': A character (or an integer from 1 to 255).

Required condition: The file must exist on disk. If it doesn't, ftok() fails and returns -1.

key_t: A 32-bit signed integer.
```

- After we have the key, we can use msgget(), semget(), shmget() to gain the IPC ID and use it to access to the object.

Example with Shared Memory

```
int shmid = shmget(key, 4096, IPC_CREAT | 0666);

shmid: The ID that the process uses to interact with that shared memory region.
```

2.2 Permission Structure

The associated data structure for all three IPC mechanisms, ipc_perm is a structure containing information that the kernel uses to manage the ownership and access control of an IPC object:

```
struct ipc_perm {
    key_t          __key;          /* Key, as supplied to 'get' call */
    uid_t          uid;            /* Owner's user ID */
    gid_t          gid;            /* Owner's group ID */
    uid_t          cuid;           /* Creator's user ID */
    gid_t          cgid;           /* Creator's group ID */
    unsigned short mode;           /* Permissions */
    unsigned short __seq;          /* Sequence number */
};
```

Code

```
nt main() {
    // 1. Create Key and IPC Shared Memory
    key_t key = ftok("/tmp/myfile", 'A');
    int shmid = shmget(key, 1024, IPC_CREAT | 0666);

    // 2. Use struct shmid_ds to get information from Kernel
    struct shmid_ds buf;
    shmctl(shmid, IPC_STAT, &buf);

    // 3. Access struct ipc_perm inside shmid_ds
    struct ipc_perm perm = buf.shm_perm;

    printf("Key: 0x%x\n", perm.__key);
    printf("Owner UID: %d\n", perm.uid);
    printf("Owner GID: %d\n", perm.gid);
    printf("Creator UID: %d\n", perm.cuid);
    printf("Creator GID: %d\n", perm.cgid);
    printf("Permissions (mode): %o\n", perm.mode);
    return 0;
}
```

Output

```
Key: 0x41060063
Owner UID: 1000
Owner GID: 1000
Creator UID: 1000
Creator GID: 1000
Permissions (mode): 666
```

Note

Permissions (mode): 666 means every user on the system has read and write access to this Shared Memory segment, but no execute access.

```
First digit (6): Owner has Read and Write access.

Second digit (6): Group has Read and Write access.

Third digit (6): Others (everyone else) have Read and Write access.
```

2.3 Configuration Limits

command: ipcs -l

```
------ Messages Limits --------
max queues system wide = 32000
max size of message (bytes) = 8192
default max size of queue (bytes) = 16384

------ Shared Memory Limits --------
max number of segments = 4096
max seg size (kbytes) = 18014398509465599
max total shared memory (kbytes) = 18446744073709551612
min seg size (bytes) = 1

------ Semaphore Limits --------
max number of arrays = 32000
max semaphores per array = 32000
max semaphores system wide = 1024000000
max ops per semop call = 500
semaphore max value = 32767
```

3. Message queue (System V)

Key Problems Solved by Message Queues:

- **Structured Data**: Unlike Pipes/FIFOs which send raw byte streams, Message Queues allow sending formatted messages with a distinct type (mtype) and payload.

- **Message Filtering & Priority**: Receivers can fetch messages based on their type (mtype) rather than strictly following First-In, First-Out (FIFO) order.
- **Messages**: Stored in Kernel memory. A sender can push a message and terminate; the message remains in the Kernel until a receiver reads it.

Messages Limits

```
Max queues system wide: 32,000


Max size of message (bytes): 8,192


Default max size of queue (bytes): 16,384
```

Note

These limits are configurable, but setting them too high is not recommended as System V Message Queues consume Kernel RAM.

API

- Creating or Opening a Message Queue:

```
int msgget(key_t key, int msgflg);
```

msgflg = IPC_CREAT: Creates a new queue if it does not exist. If it already exists, opens and connects to the existing queue.

msgflg = IPC_EXCL: Fails immediately with an error if the queue already exists. Ensures the process is the sole creator and prevents interacting with stale queues.

- Message Structure

```
struct mymsg {
    long mtype;    /* Message type (> 0) */
    char mtext[1]; /* Message body/data */
};
```

- Sending Messages:

```
int msgsnd(int msqid, const void *msgp, size_t msgsz, int msgflg);
```

msgflg = 0 (Blocking): If the queue is full, the sender blocks (sleeps) with 0% CPU usage until space becomes available.

msgflg = IPC_NOWAIT (Non-blocking): Returns -1 immediately and sets errno = EAGAIN if the queue is full.

- Receiving Messages:

```
ssize_t msgrcv(int msqid, void *msgp, size_t maxmsgsz, long msgtyp, int msgflg);
```

Reads and removes a message from the queue into msgp. maxmsgsz specifies the maximum size of mtext.

`msgtyp` Value	Retrieval Strategy	Behavior
`msgtyp == 0`	Standard FIFO	Reads the first available message in the queue.
`msgtyp > 0`	Matching Type	Reads the first message where `mtype == msgtyp`. This allows multiple processes to share a single queue without conflict. For example, `msgtyp` can be set to a PID.
`msgtyp < 0`	Priority Queue	Reads the first message with the lowest `mtype` that is less than or equal to `

Example: msgtyp < 0:

Queue [type 300, type 100, type 200, type 400, type 100], calling msgrcv(..., msgtyp = -300) repeatedly fetches messages in order: type 100, type 100, type 200, type 300. The type 400 message is skipped

- **Message Queue Control Operations**

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

cmd = IPC_RMID: Instantly destroys the message queue and its msqid_ds structure. All remaining messages are lost, and any blocked readers/writers wake up immediately with error EIDRM. (The buf argument is ignored).

cmd = IPC_SET: Updates specific fields in msqid_ds (e.g., permissions, max queue bytes) using values from buf

- **Example**

Writer

```
struct message {
   long mtype;
```

```
    char mtext[100];
};
int main() {
    // Generate IPC key using current directory '.' and project ID 'A'
    key_t key = ftok(".", 'A');
    if (key == -1) {
        perror("ftok error");
        exit(EXIT_FAILURE);
    }

    // Create or open message queue
    int msgid = msgget(key, 0666 | IPC_CREAT);
    if (msgid == -1) {
        perror("msgget error");
        exit(EXIT_FAILURE);
    }
    struct message msg;
    // --- MESSAGE 1: mtype = 1 ---
    msg.mtype = 1;
    strcpy(msg.mtext, "Message Type 1 (Normal)");
    if (msgsnd(msgid, &msg, sizeof(msg.mtext), 0) == -1) {
        perror("msgsnd 1 error");
    }
    // --- MESSAGE 2: mtype = 2 ---
    msg.mtype = 2;
    strcpy(msg.mtext, "Message Type 2 (Warning)");
    if (msgsnd(msgid, &msg, sizeof(msg.mtext), 0) == -1) {
        perror("msgsnd 2 error");
    }
    // --- MESSAGE 3: mtype = 3 ---
    msg.mtype = 3;
    strcpy(msg.mtext, "Message Type 3 (Urgent)");
    if (msgsnd(msgid, &msg, sizeof(msg.mtext), 0) == -1) {
        perror("msgsnd 3 error");
    }
    printf("Successfully sent 3 messages (mtype 1, 2, 3) to Queue!\n");
    printf("Key: 0x%08x | Queue ID: %d\n", key, msgid);
    return 0;
}
```

Output

```
-  ./app
Successfully sent 3 messages (mtype 1, 2, 3) to Queue!
Key: 0x41060722 | Queue ID: 8

- ipcs -q
------ Message Queues --------
key                  msqid      owner      perms      used-bytes   messages
0x41060722            8         dat        666            300          3
```

Reader

```
struct message {
   long mtype;
   char mtext[100];
};
int main() {
   // Get key from current folder '.' and project ID 'A'
   key_t key = ftok(".", 'A');
   int msgid = msgget(key, 0666);


   if (msgid == -1) {
       perror("Queue not found");
       exit(EXIT_FAILURE);
   }


   struct message msg;
   int count = 0;
   printf("--- START READING MESSAGES ---\n");
   // Loop until no messages left
   // IPC_NOWAIT: returns -1 (errno == ENOMSG) immediately when queue is empty
   while (msgrcv(msgid, &msg, sizeof(msg.mtext), 0, IPC_NOWAIT) != -1) {
       count++;
       printf("Message %d [type=%ld]: '%s'\n", count, msg.mtype, msg.mtext);
   }
   printf("--- Read all %d messages ---\n", count);
```

```
    // Delete the queue after reading all messages
    if (msgctl(msgid, IPC_RMID, NULL) == 0) {
        printf("Deleted queue ID %d successfully!\n", msgid);
    }
    return 0;
}
```

Output

```
- ./app
--- START READING MESSAGES ---
Message 1 [type=1]: 'Message Type 1 (Normal)'
Message 2 [type=2]: 'Message Type 2 (Warning)'
Message 3 [type=3]: 'Message Type 3 (Urgent)'
--- Read all 3 messages ---
Deleted queue ID 13 successfully!
```

4. Semaphores (System V) and POSIX Semaphores

Purpose: Synchronization mechanism used to control access to shared resources and prevent race conditions between processes.

Types:

- Binary Semaphore: Value is either 0 or 1 (acts like a Mutex/Lock).Counting Semaphore:
- Value is >= 1 (tracks available resources, e.g., buffer slots).

4.1 Semaphores (System V)

Unlike System V Shared Memory or Message Queues which manage single objects, System V Semaphores operate as Semaphore Sets (an array of semaphores allocated together in Kernel memory).

```
------ Semaphore Limits --------
max number of arrays = 32000
max semaphores per array = 32000
max semaphores system wide = 1024000000
max ops per semop call = 500
semaphore max value = 32767
```

Concept	System V Semaphores
Allocation	Sets (arrays) of semaphores allocated per key.
Atomic Operations	`semop()` can perform operations on multiple semaphores in a set atomically.
Undo Mechanism	`SEM_UNDO` automatically reverts operations if a process crashes while holding a lock.

API

- Creation & Access

```
int semget(key_t key, int nsems, int semflg);
```

nsems: Number of semaphores in the set.

semflg: Permissions + flags (IPC_CREAT, IPC_EXCL).

- A data structure used to describe an operation on a System V semaphore.

```
struct sembuf {
    unsigned short sem_num; /* Semaphore index in set (0 to nsems-1) */
    short          sem_op;  /* Operation: < 0 (Wait/Lock), > 0 (Signal/Unlock), 0 (Wait-for-
Zero) */
    short          sem_flg; /* Flags: IPC_NOWAIT, SEM_UNDO */
};
```

`sem_op` Value	Operation	Action
`sem_op < 0`	**Wait / Decrement**	Decrements the semaphore value. If the operation cannot be completed immediately, the process blocks until the semaphore value is large enough.
`sem_op > 0`	**Signal / Increment**	Increments the semaphore value and may wake up waiting processes.
`sem_op == 0`	**Wait for Zero**	Blocks until the semaphore value becomes exactly `0`.

- Lock operation (Wait / P operation)

```
void sem_lock(int semid) {
    struct sembuf sb = {
        .sem_num = sem_num,        // Select semaphore at index in the set
        .sem_op  = -1,             // Decrement value (Wait / Lock)
        .sem_flg = SEM_UNDO        // Auto-revert operation if process terminates unexpectedly
```

```
    };
    semop(semid, &sb, 1);
}
```

- Unlock operation (Signal / V operation)

```
void sem_unlock(int semid) {
    struct sembuf sb = {
        .sem_num = sem_num,        // Select semaphore at index 0 in the set
        .sem_op  = 1,              // Increment value (Signal / Unlock)
        .sem_flg = SEM_UNDO        // Auto-revert operation if process terminates unexpectedly
    };
    semop(semid, &sb, 1);
}
```

- Control Operations

```
int semctl(int semid, int semnum, int cmd, union semun arg);
```

```
Argument **cmd**
|
├── 1. Set Management
|   ├── IPC_RMID : Destroys the entire Semaphore Set from Kernel memory.
|   ├── IPC_STAT : Retrieves metadata and status of the Semaphore Set into a struct.
|   └── IPC_SET  : Updates configurable attributes (e.g., permissions) of the Semaphore Set.
|
├── 2. Read / Write Semaphore Values
|   ├── GETVAL : Returns an integer representing the current value (available units) of a
single semaphore.
|   ├── SETVAL : Sets the value for a single semaphore in the set.
|   ├── GETALL : Reads the current values of ALL semaphores in the set into an array.
|   └── SETALL : Sets the values of ALL semaphores in the set simultaneously.
|
└── 3. Process & Wait Queue Information
    ├── GETPID  : Returns the Process ID (PID) of the LAST process that performed an operation
on the semaphore.
    ├── GETNCNT : Returns the count of processes currently waiting for the semaphore value to
INCREASE (waiting for resources).
    └── GETZCNT : Returns the count of processes currently waiting for the semaphore value to
reach ZERO.
```

Code exmaple

```
union semun {
   int val;
   struct semid_ds *buf;
   unsigned short *array;
};

// Define index numbers for 3 semaphores in the set
#define SEM_PRINTER 0   // Semaphore 0: Printer
#define SEM_SCANNER 1   // Semaphore 1: Scanner
#define SEM_PROJECTOR 2 // Semaphore 2: Projector

// Function to Lock (Wait / P operation) a specific semaphore in the set
void sem_lock(int semid, int sem_num) {
   struct sembuf sb = {
       .sem_num = sem_num,   // Choose index (0, 1, or 2)
       .sem_op  = -1,        // Request resource
       .sem_flg = SEM_UNDO
   };
   if (semop(semid, &sb, 1) == -1) {
       perror("Error: semop lock failed");
       exit(1);
   }
}

// Function to Unlock (Signal / V operation) a specific semaphore in the set
void sem_unlock(int semid, int sem_num) {
   struct sembuf sb = {
       .sem_num = sem_num,   // Choose index (0, 1, or 2)
       .sem_op  = 1,         // Release resource
       .sem_flg = SEM_UNDO
   };
   if (semop(semid, &sb, 1) == -1) {
       perror("Error: semop unlock failed");
       exit(1);
   }
}
```

```
int main() {
   key_t key = ftok(".", 65);

   // 1. Create a set of 3 semaphores (nsems = 3)
   int sem_id = semget(key, 3, 0666 | IPC_CREAT);
   if (sem_id == -1) {
       perror("Error: semget failed");
       exit(1);
   }

   // 2. Set initial values for all 3 semaphores using SETALL
   // All set to 1 (ready to use)
   unsigned short init_values[3] = {1, 1, 1};
   union semun arg;
   arg.array = init_values;

   if (semctl(sem_id, 0, SETALL, arg) == -1) {
       perror("Error: semctl SETALL failed");
       exit(1);
   }

    printf("Semaphore set created successfully.\n");
    printf("Semaphore ID: %d\n", sem_id);
    printf("Press Enter to continue...\n");

   pid_t pid = fork();

   if (pid < 0) {
       perror("Error: fork failed");
       exit(1);
   } else if (pid == 0) {
       // --- CHILD PROCESS: Uses Printer (Sem 1) and Projector (Sem 2) ---

       printf("[Child] Waiting for SCANNER...\n");
       sem_lock(sem_id, SEM_SCANNER);
       printf("[Child] --> Got SCANNER!\n");

       printf("[Child] Waiting for PROJECTOR...\n");
       sem_lock(sem_id, SEM_PROJECTOR);
```

```
        printf("[Child] --> Got PROJECTOR!\n");

        printf("[Child] Releasing SCANNER and PROJECTOR.\n");
        sem_unlock(sem_id, SEM_SCANNER);
        sem_unlock(sem_id, SEM_PROJECTOR);

    } else {
        // --- PARENT PROCESS: Uses Scanner (Sem 1) and Printer (Sem 0) ---

        printf("[Parent] Waiting for SCANNER...\n");
        sem_lock(sem_id, SEM_SCANNER);
        printf("[Parent] --> Got SCANNER!\n");

        printf("[Parent] Waiting for PRINTER...\n");
        sem_lock(sem_id, SEM_PRINTER); // Will block if Child holds the Printer
        printf("[Parent] --> Got PRINTER!\n");

        sleep(2); // Simulate work

        printf("[Parent] Releasing SCANNER and PRINTER.\n");
        sem_unlock(sem_id, SEM_SCANNER);
        sem_unlock(sem_id, SEM_PRINTER);

        wait(NULL); // Wait for child process to finish

        // 3. Remove all 3 semaphores from the system
        if (semctl(sem_id, 0, IPC_RMID, arg) == -1) {
            perror("Error: semctl remove failed");
            exit(1);
        }
        printf("Cleaned up and deleted semaphore set successfully.\n");
    }
    return 0;
}
```

Output

```
Semaphore set created successfully.
Semaphore ID: 11
Press Enter to continue...
```

```
[Parent] Waiting for SCANNER...
[Parent] --> Got SCANNER!
[Parent] Waiting for PRINTER...
[Parent] --> Got PRINTER!
[Child] Waiting for SCANNER...

//child waits for 2 seconds for parent to release the lock SCANNER

[Parent] Releasing SCANNER and PRINTER.
[Child] --> Got SCANNER!
[Child] Waiting for PROJECTOR...
[Child] --> Got PROJECTOR!
[Child] Releasing SCANNER and PROJECTOR.
Cleaned up and deleted semaphore set successfully.
```

4.2 POSIX Semaphores

POSIX Semaphores are divided into two main types:

Named Semaphores

- Use Case: Used for synchronization between independent or parent-child processes.
- Storage: Identified by a name string and represented as virtual files in /dev/shm/.

Unnamed

- Use Case: Used primarily between threads within the same process, or parent-child processes sharing a memory region (Shared Memory).
- Storage: Allocated directly in user space memory (e.g., global variables or shared memory segments).

API

Feature	Named Semaphore	Unnamed Semaphore
Identification	Unique path name (e.g., `/my_sem`)	Memory address (`sem_t` variable)
Primary Scope	Unrelated / independent processes	Threads or related processes using shared memory
Creation API	`sem_open()`	`sem_init()`
Destruction API	`sem_close()` + `sem_unlink()`	`sem_destroy()`
Location	`/dev/shm/sem.name`	Stack, heap, or shared memory segment

Lock / Unlock Operation

Operation	POSIX Function	Meaning
Wait (P / Lock)	`sem_wait(sem_t *sem);`	Decreases the semaphore value by 1. If the value is `0`, the process blocks and waits.
Try Lock	`sem_trywait(sem_t *sem);`	Decreases the value by 1. If the value is `0`, it does not wait and returns `EAGAIN` immediately.
Signal (V / Unlock)	`sem_post(sem_t *sem);`	Increases the semaphore value by 1 and wakes up a waiting process, if any.
Get Value	`sem_getvalue(sem, &val);`	Gets the current semaphore value and stores it in the variable `val`.

Code: Named Semaphore

```
// Semaphore names (Must start with '/')
#define SEM_PRINTER   "/sem_printer"
#define SEM_SCANNER   "/sem_scanner"
#define SEM_PROJECTOR "/sem_projector"

int main() {
   // 1. Create and initialize 3 separate POSIX Named Semaphores (Initial value = 1)
   sem_t *sem_printer   = sem_open(SEM_PRINTER,   O_CREAT, 0666, 1);
   sem_t *sem_scanner   = sem_open(SEM_SCANNER,   O_CREAT, 0666, 1);
   sem_t *sem_projector = sem_open(SEM_PROJECTOR, O_CREAT, 0666, 1);

   if (sem_printer == SEM_FAILED || sem_scanner == SEM_FAILED || sem_projector == SEM_FAILED)
{
        perror("Error: sem_open failed");
        exit(1);
   }

   pid_t pid = fork();

   if (pid < 0) {
        perror("Error: fork failed");
        exit(1);
   } else if (pid == 0) {
        // --- CHILD PROCESS ---
        printf("[Child] Waiting for SCANNER...\n");
```

```
    sem_wait(sem_scanner); // Lock Printer
    printf("[Child] --> Got SCANNER!\n");

    printf("[Child] Waiting for PROJECTOR...\n");
    sem_wait(sem_projector); // Lock Projector
    printf("[Child] --> Got PROJECTOR!\n");

    printf("[Child] Releasing SCANNER and PROJECTOR.\n");
    sem_post(sem_scanner);   // Unlock Scanner
    sem_post(sem_projector); // Unlock Projector

    // Close semaphores in child process
    sem_close(sem_printer);
    sem_close(sem_scanner);
    sem_close(sem_projector);

} else {
    // --- PARENT PROCESS ---
    printf("[Parent] Waiting for SCANNER...\n");
    sem_wait(sem_scanner); // Lock Scanner
    printf("[Parent] --> Got SCANNER!\n");

    printf("[Parent] Waiting for PRINTER...\n");
    sem_wait(sem_printer); // Lock Printer (Will block if Child holds it)
    printf("[Parent] --> Got PRINTER!\n");

    sleep(2); // Simulate work

    printf("[Parent] Releasing SCANNER and PRINTER.\n");
    sem_post(sem_scanner); // Unlock Scanner
    sem_post(sem_printer); // Unlock Printer

    wait(NULL); // Wait for child process to finish

    // 2. Clean up and remove semaphores from the system
    sem_close(sem_printer);
    sem_close(sem_scanner);
    sem_close(sem_projector);

    sem_unlink(SEM_PRINTER);
```

```
        sem_unlink(SEM_SCANNER);
        sem_unlink(SEM_PROJECTOR);

        printf("Cleaned up and unlinked all POSIX semaphores successfully.\n");
    }

    return 0;
}
```

Output

```
- ls -l /dev/shm/sem.*

rw-rw-r-- 1 dat dat 32 Sep 16 15:41 /dev/shm/sem.sem_printer
rw-rw-r-- 1 dat dat 32 Sep 16 15:41 /dev/shm/sem.sem_projector
rw-rw-r-- 1 dat dat 32 Sep 16 15:41 /dev/shm/sem.sem_scanner

- ./app
[Parent] Waiting for SCANNER...
[Parent] --> Got SCANNER!
[Parent] Waiting for PRINTER...
[Parent] --> Got PRINTER!
[Child] Waiting for SCANNER...

//child waits for 2 seconds for parent to release the lock SCANNER

[Parent] Releasing SCANNER and PRINTER.
[Child] --> Got SCANNER!
[Child] Waiting for PROJECTOR...
[Child] --> Got PROJECTOR!
[Child] Releasing SCANNER and PROJECTOR.
Cleaned up and deleted semaphore set successfully.
```

4.3 Compare Semaphores (System V) and POSIX Semaphores

Feature / Aspect	System V Semaphores (Legacy)	POSIX Semaphores (Modern Standard)

Standardization	Older UNIX IPC mechanism, introduced with System V.	Defined by the POSIX standard (POSIX.1b / real-time extensions).
API Complexity	More complex and verbose (`semget`, `semctl`, `semop`).	Simpler API (`sem_init`, `sem_destroy`, `sem_wait`, `sem_post`, `sem_open`, `sem_close`, etc.).
Data Unit	Operates on **Sets of Semaphores** (an array of semaphore values).	Each semaphore is an **individual counter**.
Atomic Multi-Operations	**Supported:** `semop()` can atomically perform multiple operations on multiple semaphores in a set.	**Not supported:** `sem_wait()` and `sem_post()` operate on one semaphore at a time.
Performance & Overhead	Generally has more kernel-management overhead due to its System V IPC design.	Often simpler and lighter to use, especially for thread synchronization. Actual performance depends on the implementation and workload.
Scope / Flexibility	System V IPC mechanism designed for inter-process synchronization.	Supports **unnamed semaphores** (`sem_init`) for threads/processes sharing memory and **named semaphores** (`sem_open`) for unrelated processes.
Auto-Cleanup on Crash	**Supports `SEM_UNDO`:** semaphore operations can be automatically adjusted if a process terminates unexpectedly.	**No equivalent to `SEM_UNDO`:** if a process terminates while logically holding a semaphore, the semaphore value is not automatically restored.

5. Shared Memory (System V) and Client/Server Properties

5.1 Shared Memory (System V)

System V Shared Memory allows two or more processes to share a specific physical memory segment.

- Fastest IPC Mechanism: Data written by one process is instantly accessible to others without Kernel intervention or system call overhead (unlike Pipes or Message Queues, which require intermediate kernel-space buffering).
- Synchronization Required: prevent race conditions during concurrent read/write operations.

```
------ Shared Memory Limits --------
max number of segments = 4096
max seg size (kbytes) = 18014398509465599
max total shared memory (kbytes) = 18446744073709551612
```

```
min seg size (bytes) = 1
```

5 Steps to Use System V Shared Memory

- **Step 1. Create / Get Segment:** `shmget()`

```
int shmget(key_t key, size_t size, int shmflg);
```

Allocates a new shared memory segment or retrieves the ID of an existing one.

When a new segment is created, its contents are automatically initialized to 0.

- **2. Attach Segment: shmat()**

```
void *shmat(int shmid, const void *shmaddr, int shmflg);
```

Attaches the shared memory segment to the calling process's address space.

Returns a pointer (void *) to the shared memory segment.

- **3. Read / Write Data**

After attaching the segment, processes can access the shared memory directly using standard C pointer operations.

- **4. Detach Segment: shmdt()**

```
int shmdt(const void *shmaddr);
```

Detaches the shared memory segment from the calling process's virtual address space.

The shared memory segment itself is not deleted.

- **5. Control / Destroy Segment: shmctl()**

```
int shmctl(int shmid, int cmd, struct shmid_ds *buf);
```

Used to control the shared memory segment.

Common commands:

- IPC_RMID → Mark the segment for deletion.
- IPC_STAT → Get information about the segment.
- IPC_SET → Update some metadata of the segment.

Code example Shared memory + Semaphore

Idea:

```
         Shared Memory
        +-------------+
        |     10      |  ← Process A writes
        |             |
        |     20      |  ← Process B modifies
        +-------------+
               ↑
               |
           Semaphore
            value = 0
               |
         A waits here
               |
         B modifies data
               |
         B sem_unlock()
               |
         semaphore = 1
               |
         A continues
               ↓
         A reads new data
```

code A

```
union semun {
   int val;
   struct semid_ds *buf;
   unsigned short *array;
};

// Function to read and print details of struct shmid_ds
void print_shm_ds(int shmid, const char *label) {
   struct shmid_ds ds;
   if (shmctl(shmid, IPC_STAT, &ds) == -1) {
       perror("shmctl IPC_STAT error");
       return;
   }
```

```
    printf("\n--- [CHECK STRUCT SHMID_DS: %s] ---\n", label);
    printf("Segment Size      : %zu bytes\n", ds.shm_segsz);
    printf("Attached Processes: %ld\n", (long)ds.shm_nattch);
    printf("Creator PID       : %d\n", ds.shm_cpid);
    printf("Last Process PID  : %d\n", ds.shm_lpid);
    printf("Permissions       : 0%o\n", ds.shm_perm.mode);
    if (ds.shm_atime > 0) printf("Last Attach Time  : %s", ctime(&ds.shm_atime));
    if (ds.shm_dtime > 0) printf("Last Detach Time  : %s", ctime(&ds.shm_dtime));
    printf("-----------------------------------------\n\n");
}

// Lock function (Wait / P operation) - Decrement Semaphore
void sem_lock(int semid) {
    struct sembuf sb = {
        .sem_num = 0,
        .sem_op = -1,
        .sem_flg = SEM_UNDO
    };
    semop(semid, &sb, 1);
}

// Unlock function (Signal / V operation) - Increment Semaphore
void sem_unlock(int semid) {
    struct sembuf sb = {
        .sem_num = 0,
        .sem_op = 1,
        .sem_flg = SEM_UNDO
    };
    semop(semid, &sb, 1);
}

int main() {
    // 1. Create Keys
    key_t shm_key = ftok("A_share_semaphore.c", 65);
    key_t sem_key = ftok("A_share_semaphore.c", 66);
    if (shm_key == -1 || sem_key == -1) {
        perror("ftok error");
        exit(1);
    }
```

```
// 2. Create Shared Memory (1 integer)
int shmid = shmget(shm_key, sizeof(int), 0666 | IPC_CREAT);
if (shmid == -1) {
    perror("shmget error");
    exit(1);
}

// 3. Create and initialize Semaphore
int semid = semget(sem_key, 1, 0666 | IPC_CREAT);
if (semid == -1) {
    perror("semget error");
    exit(1);
}

// Set initial Semaphore value = 0 (A will wait for B)
union semun arg;
arg.val = 0;
if (semctl(semid, 0, SETVAL, arg) == -1) {
    perror("semctl SETVAL error");
    exit(1);
}

printf("[Process A] SHMID: %d | SEMID: %d\n", shmid, semid);

// 4. Attach Shared Memory
int *num = (int *)shmat(shmid, NULL, 0);
if (num == (int *)-1) {
    perror("shmat error");
    exit(1);
}
// 5. Write initial data
*num = 10;
printf("[Process A] Initial number written: %d\n", *num);
print_shm_ds(shmid, "After Process A attached & wrote data");
printf("[Process A] Waiting for Process B to modify data...\n");
// 6. WAIT FOR SIGNAL FROM PROCESS B (Wait until Semaphore > 0)
sem_lock(semid);
print_shm_ds(shmid, "After Process B modified data");
```

```
    // 7. Read modified data
    printf("[Process A] Read modified number: %d\n", *num);

    // 8. Clean up resources
    shmdt(num);
    shmctl(shmid, IPC_RMID, NULL); // Remove Shared Memory
    semctl(semid, 0, IPC_RMID);    // Remove Semaphore
    printf("[Process A] Memory & Semaphore cleaned up. Done.\n");

    return 0;
}
```

Code B

```
// Function to read and print details of struct shmid_ds
void print_shm_ds(int shmid, const char *label) {
    struct shmid_ds ds;
    if (shmctl(shmid, IPC_STAT, &ds) == -1) {
        perror("shmctl IPC_STAT error");
        return;
    }

    printf("\n--- [CHECK STRUCT SHMID_DS: %s] ---\n", label);
    printf("Segment Size      : %zu bytes\n", ds.shm_segsz);
    printf("Attached Processes: %ld\n", (long)ds.shm_nattch);
    printf("Creator PID       : %d\n", ds.shm_cpid);
    printf("Last Process PID  : %d\n", ds.shm_lpid);
    printf("Permissions       : 0%o\n", ds.shm_perm.mode);
    if (ds.shm_atime > 0) printf("Last Attach Time  : %s", ctime(&ds.shm_atime));
    if (ds.shm_dtime > 0) printf("Last Detach Time  : %s", ctime(&ds.shm_dtime));
    printf("-----------------------------------------\n\n");
}

// Unlock function (Signal / V operation) - Increment Semaphore
void sem_unlock(int semid) {
    struct sembuf sb = {
        .sem_num = 0,
        .sem_op = 1,
        .sem_flg = SEM_UNDO
```

```
    };
    semop(semid, &sb, 1);
}

int main() {
    // 1. Get same keys as Process A
    key_t shm_key = ftok("A_share_semaphore.c", 65);
    key_t sem_key = ftok("A_share_semaphore.c", 66);
    if (shm_key == -1 || sem_key == -1) {
        perror("ftok error");
        exit(1);
    }

    // 2. Open existing Shared Memory and Semaphore
    int shmid = shmget(shm_key, sizeof(int), 0666);
    int semid = semget(sem_key, 1, 0666);
    if (shmid == -1 || semid == -1) {
        perror("IPC locate error (Is Process A running?)");
        exit(1);
    }

    printf("[Process B] SHMID: %d | SEMID: %d\n", shmid, semid);

    // 3. Attach Shared Memory
    int *num = (int *)shmat(shmid, NULL, 0);
    if (num == (int *)-1) {
        perror("shmat error");
        exit(1);
    }

    print_shm_ds(shmid, "While B is attached");

    // 4. Read and modify data (10 -> 15)
    printf("[Process B] Read number: %d\n", *num);
    *num += 5;
    printf("[Process B] Modified number to: %d\n", *num);

    // 5. SEND SIGNAL TO UNBLOCK PROCESS A
    sem_unlock(semid);
```

```
    // 6. Detach memory
    shmdt(num);
    printf("[Process B] Signal sent. Detached. Done.\n");

    return 0;
}
```

Output

```
- Terminal 1: ./A
[Process A] SHMID: 163840 | SEMID: 7
[Process A] Initial number written: 10
--- [CHECK STRUCT SHMID_DS: After Process A attached & wrote data] ---
Segment Size      : 4 bytes
Attached Processes: 1
Creator PID       : 17155
Last Process PID  : 17155
Permissions       : 0666
Last Attach Time  : Tue Sep 15 15:53:28 2026
----------------------------------------
[Process A] Waiting for Process B to modify data…

- Terminal 2:  ./B
[Process B] SHMID: 163840 | SEMID: 7
--- [CHECK STRUCT SHMID_DS: While B is attached] ---
Segment Size      : 4 bytes
Attached Processes: 2
Creator PID       : 17155
Last Process PID  : 17354
Permissions       : 0666
Last Attach Time  : Tue Sep 15 15:55:49 2026
----------------------------------------
[Process B] Read number: 10
[Process B] Modified number to: 15
[Process B] Signal sent. Detached. Done.

- Terminal 1: Continue
--- [CHECK STRUCT SHMID_DS: After Process B modified data] ---
Segment Size      : 4 bytes
```

```
Attached Processes: 1
Creator PID       : 17155
Last Process PID  : 17354
Permissions       : 0666
Last Attach Time  : Tue Sep 15 15:55:49 2026
Last Detach Time  : Tue Sep 15 15:55:49 2026
----------------------------------------
[Process A] Read modified number: 15
[Process A] Memory & Semaphore cleaned up. Done.
```

5.2 Client/Server properties

- **Concept**

Client/Server = Communication Model

Client: A process that asks for a service.

Server: A process that provides the service and handles requests.

```
               Request
       ---------------------->
Client                        Server
       <----------------------
              Response
```

Client sends a request.

Server processes it and sends back a response.

- **Role separation**

Client and Server have different roles

```
Client                    Server
------                    ------
Request                   Provide service
Send request        --->  Receive request
Receive response    <---  Process request
```

- **Note**

Server usually starts first and must exist to receive requests.

```
        Server
          |
          | waiting
          |
          | <---- Client request
          |
          | process
          |
          | ----> response
```

Client is usually the one that starts the communication.

```
        Client
          |
          | request
          v
        Server
```

Server can serve multiple Clients

```
            ┌── Client 1
            │
            ├── Client 2
  Server ───|
            ├── Client 3
            │
            └── Client 4
```

- **Client/Server + IPC**

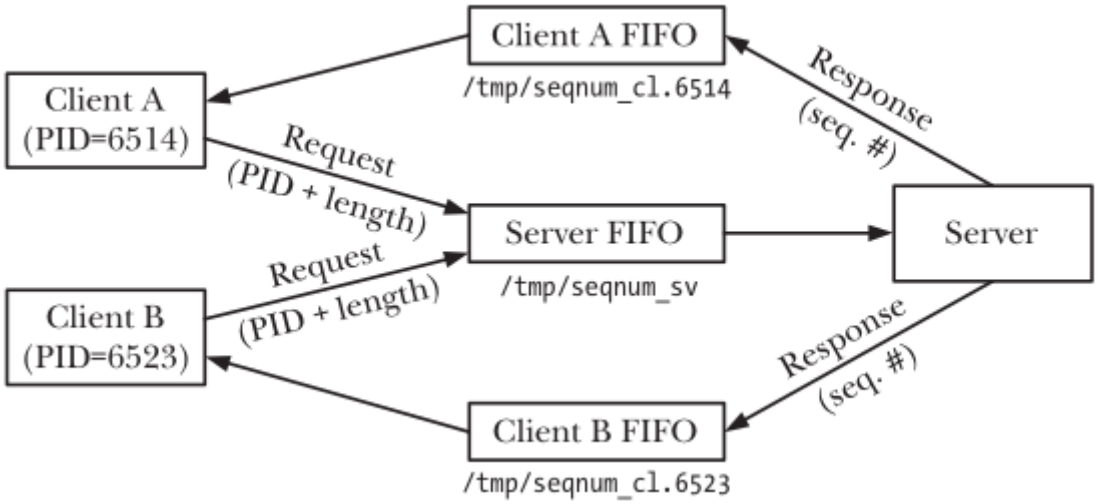
Client/Server is a communication model.

FIFO, Message Queue, and Shared Memory are IPC mechanisms that can be used to implement that communication.

```
            Client/Server
            |
            ├── FIFO
```

```
        ├── Message Queue
        └── Shared Memory
```

Example FIFO



Code

- Serve

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

// File paths for FIFOs
#define SERVER_FIFO "/tmp/seqnum_sv"
#define CLIENT_FIFO_TEMPLATE "/tmp/seqnum_cl.%d"

// Data structure for client request
typedef struct {
    pid_t client_pid;
    int length;
} request_t;

// Data structure for server response
typedef struct {
    int start_seqNum;
```

```
} response_t;

int main() {
    int server_fd, client_fd;
    request_t req;
    response_t resp;
    int current_seqNum = 1000;

    // Step 1: Create the main server FIFO pipe
    mkfifo(SERVER_FIFO, 0666);
    printf("[SERVER] Listening on %s...\n", SERVER_FIFO);

    // Step 2: Open server FIFO in read-write mode to keep it open
    server_fd = open(SERVER_FIFO, O_RDWR);

    // Step 3: Run forever to process incoming requests
    while (1) {
        // Read a request from any connected client
        ssize_t bytes_read = read(server_fd, &req, sizeof(request_t));
        if (bytes_read > 0) {
            printf("\n[SERVER] Received request from Client PID = %d (Requesting %d
numbers)\n",
                   req.client_pid, req.length);

            // Calculate the response data
            resp.start_seqNum = current_seqNum;
            current_seqNum += req.length;

            // Build the specific client FIFO file path using its PID
            char client_fifo_path[64];
            snprintf(client_fifo_path, sizeof(client_fifo_path), CLIENT_FIFO_TEMPLATE,
req.client_pid);

            // Step 4: Open the client FIFO and send back the response
            client_fd = open(client_fifo_path, O_WRONLY);
            if (client_fd != -1) {
                write(client_fd, &resp, sizeof(response_t));
                close(client_fd);
                printf("[SERVER] Successfully responded to %s!\n", client_fifo_path);
            } else {
```

```
                perror("[SERVER] Error opening Client FIFO");
            }
        }
    }

    // Step 5: Clean up resources on exit
    close(server_fd);
    unlink(SERVER_FIFO);
    return 0;
}
```

- Client

```
// File paths for FIFOs
#define SERVER_FIFO "/tmp/seqnum_sv"
#define CLIENT_FIFO_TEMPLATE "/tmp/seqnum_cl.%d"

// Data structure for client request
typedef struct {
    pid_t client_pid;
    int length;
} request_t;

// Data structure for server response
typedef struct {
    int start_seqNum;
} response_t;

int main(int argc, char *argv[]) {
    // Check command line arguments
    if (argc < 2) {
        printf("Usage: %s <requested_seqnum_count>\n", argv[0]);
        return 1;
    }

    int req_length = atoi(argv[1]);
    pid_t my_pid = getpid();

    // Step 1: Create a private Client FIFO using process PID
    char my_fifo_path[64];
```

```
    snprintf(my_fifo_path, sizeof(my_fifo_path), CLIENT_FIFO_TEMPLATE, my_pid);
    mkfifo(my_fifo_path, 0666);

    printf("[CLIENT PID=%d] Successfully created private FIFO: %s\n", my_pid, my_fifo_path);

    // Step 2: Send request data (PID + Length) to Server FIFO
    request_t req = { .client_pid = my_pid, .length = req_length };
    int server_fd = open(SERVER_FIFO, O_WRONLY);
    if (server_fd == -1) {
        perror("[CLIENT] Failed to connect to Server FIFO (Is the server running?)");
        unlink(my_fifo_path);
        return 1;
    }
    write(server_fd, &req, sizeof(request_t));
    close(server_fd);

    // Step 3: Wait and read response from private Client FIFO
    int my_fd = open(my_fifo_path, O_RDONLY);
    response_t resp;
    read(my_fd, &resp, sizeof(response_t));
    close(my_fd);

    printf("==> [CLIENT PID=%d] RECEIVED RESULT: Sequence Number = %d\n", my_pid,
resp.start_seqNum);

    // Step 4: Delete private FIFO on exit
    unlink(my_fifo_path);
    return 0;
}
```

Output

```
./server
[SERVER] Listening on /tmp/seqnum_sv...

./client  5
[CLIENT PID=16139] Successfully created private FIFO: /tmp/seqnum_cl.16139
==> [CLIENT PID=16139] RECEIVED RESULT: Sequence Number = 1000

[SERVER] Received request from Client PID = 16139 (Requesting 5 numbers)
```

```
[SERVER] Successfully responded to /tmp/seqnum_cl.16139!

./client 10
[CLIENT PID=16236] Successfully created private FIFO: /tmp/seqnum_cl.16236
==> [CLIENT PID=16236] RECEIVED RESULT: Sequence Number = 1005

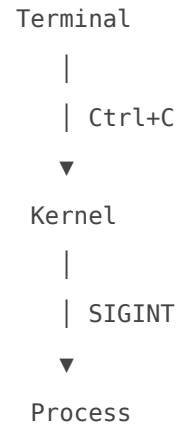
[SERVER] Received request from Client PID = 16236 (Requesting 10 numbers)
[SERVER] Successfully responded to /tmp/seqnum_cl.16236!
```

6.Signal Concepts, signal Function, Unreliable Signals, Interrupted System Calls, Reentrant Functions

A signal is an asynchronous Inter-Process Communication (IPC) mechanism. It works like a "software interrupt" sent to a process to notify it that a specific event has occurred.

Purpose: To report unusual events (for example: division by zero, invalid memory access) or to control processes (stop, continue, terminate).

Example



When run a program and then press **Ctrl+C**. The terminal does not directly kill the program. It sends **SIGINT** to the process. By default, SIGINT stops the process.

Signal handling states:

```
Default action: Perform the default action of the OS
```

```
Ignore: Ignore the signal.


Catch (Signal Handler): Catch the signal and run a custom function.
```

6.1 Signal concepts

Signal	Meaning
SIGINT	Interrupt, usually from `Ctrl+C`
SIGTERM	Request the process to stop
SIGKILL	Stop the process immediately
SIGSTOP	Pause the process
SIGCONT	Continue the paused process
SIGCHLD	Child process changed its state
SIGALRM	Timer ran out of time
SIGUSR1	User-defined signal 1
SIGUSR2	User-defined signal 2
SIGSEGV	Invalid memory access (Segmentation fault)

check more : kill -l

- **Signal lifecycle**

A signal usually goes through these states

```
            Signal generated
                  |
                  ▼
               Pending
                  |
                  ▼
              Delivered
                  |
                  ▼
        Process handles signal
```

Example:

```
kill(pid, SIGUSR1);
```

```
                Process A
                    |
                    | kill(pid_B, SIGUSR1)
                    ▼
                Kernel
                    |
                    | signal pending
                    ▼
                Process B
                    |
                    | signal delivered
                    ▼
                SIGUSR1 handler
```

6.2 signal funtion and Unreliable Signals

To set up a handler for a signal, Linux provides two functions:

- **signal() function (Classic / Simple)**

```
sighandler_t signal(int signum, sighandler_t handler);
```

code:

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig)
{
    // Async-Signal-Safe
    write(STDOUT_FILENO, "Received signal\n", 16);
}

int main()
{
    signal(SIGINT, handler);

    while (1)
    {
        printf("Running...\n");
```

```
        sleep(1);
    }

    return 0;
}
```

Output:

```
Running...
Running...
Running...
^CReceived signal 2
Running...
Running...
^CReceived signal 2
Running...
```

Drawback:

Inconsistent behavior across different UNIX systems (for example: automatically resetting the handler back to default after catching a signal).

=> Signals are considered "unreliable."

- **sigaction() function (POSIX Standard - Recommended)**

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

code:

```
void handle_sigint(int sig) {
    write(STDOUT_FILENO, "\n[sigaction()] Safely caught SIGINT!\n", 37);
}

int main() {
    struct sigaction sa;

    // Set the handler function
    sa.sa_handler = handle_sigint;

    // Clear the signal mask set
    sigemptyset(&sa.sa_mask);
```

```
    // Enable SA_RESTART flag: Automatically restart interrupted System Calls (like read,
sleep)
    sa.sa_flags = SA_RESTART;

    // Set up sigaction for SIGINT
    if (sigaction(SIGINT, &sa, NULL) < 0) {
        perror("Error setting up sigaction");
        return 1;
    }

    printf("Program is running. Press Ctrl + C to test sigaction()...\n");
    while (1) {
        sleep(1);
    }
    return 0;
}
```

Output

```
Program is running. Press Ctrl + C to test sigaction()...
^C
[sigaction()] Safely caught SIGINT!
^C
[sigaction()] Safely caught SIGINT!
^C
[sigaction()] Safely caught SIGINT!
```

- **Compare Singal and Sigaction**

Criteria	`signal()`	`sigaction()`
Consistency	Behavior differs across operating systems (e.g., System V vs. BSD).	Standardized by POSIX and behaves consistently across Linux/UNIX systems.
Handler Reset	May automatically reset the handler to `SIG_DFL` after handling one signal on older systems.	The handler remains installed for subsequent signals by default.
Signal Masking	Does not provide explicit control over which signals are blocked during the handler.	Automatically blocks the current signal while the handler runs; additional signals can be blocked using `sa_mask`.

Criteria	`signal()`	`sigaction()`
Control Flags	Does not support advanced configuration flags.	Supports flags such as `SA_RESTART` (automatically restart interrupted system calls) .

6.3 Interrupted System Calls

When a process is blocked in a "slow" System Call (such as read() from keyboard/network, write(), wait(), or pause()), if a signal arrives and triggers a Signal Handler, that System Call gets interrupted midway.

Consequence: The System Call returns -1 and sets the error code errno = EINTR.

Solution:

1. Manual handling in code:

```
while ((n = read(fd, buf, BUFSIZE)) < 0) {
    if (errno == EINTR)
        continue; // Read again if interrupted by a signal
    // Handle other errors
}
```

2. Automatic restart:

sigaction(), add the SA_RESTART flag in sa_flags. The operating system will automatically resume the interrupted System Call instead of returning EINTR.

Code Signal

```
void handle_sigint(int sig) {
    write(STDOUT_FILENO, "\n[Signal Handler] Caught SIGINT!\n", 33);
}

int main() {
    char buf[100];

    // Register signal handler using default signal()
    signal(SIGINT, handle_sigint);

    printf("Please enter a string (or press Ctrl+C to test System Call interruption):\n");

    // read() is a slow System Call (blocked)
```

```
    ssize_t n = read(STDIN_FILENO, buf, sizeof(buf) - 1);

    // WHEN PRESSING CTRL+C: read() returns -1 and errno is set to EINTR
    if (n < 0) {
        // An error occurs here because errno == EINTR is not checked!
        perror("read() error");
        printf("Program exited because read() returned an error!\n");
        return 1;
    }

    buf[n] = '\0';
    printf("You entered: %s", buf);
    return 0;
}
```

Expect

```
read() error: Interrupted system call, and the program exits immediately instead of waiting to type again.
```

Output

```
Please enter a string (or press Ctrl+C to test System Call interruption):
^C
[Signal Handler] Caught SIGINT!
a
You entered: a
```

Explain Why it still print a

Cause: glibc automatically enables SA_RESTART for signal()

```
The reality on Linux: The signal() function on Linux is not the classic System V implementation. Instead, glibc wraps the signal() function using sigaction().

Default glibc behavior: When call signal(SIGINT, handle_sigint), glibc automatically sets the SA_RESTART flag under the hood.
```

6.4 Reentrant Functions

It is a function that can be paused midway (for example, interrupted by a Signal) and called again by another execution thread (or by the Signal Handler itself) while still remaining correct and

without corrupting data.

Rules for writing a Signal Handler:

Only call "Async-Signal-Safe" functions (defined by POSIX, for example: write(), _exit(), unlink()).

DO NOT call non-reentrant functions:

- Functions using static buffers or global variables.
- Dynamic memory management functions: malloc(), free().
- Standard I/O functions in C library: printf(), scanf(), fopen() (because they use global buffers and locking mechanisms).

7. Reliable-Signal semantics: kill/raise, alarm/pause, Signal Sets, sigprocmask, sigpending, sigaction, sigsuspend

7.1 kill/raise

7.1.1 kill

kill: sends a signal to a process or a group of processes

```
int kill(pid_t pid, int signo);
```

`pid` value	Meaning
`pid > 0`	Send the signal to the process with the specified PID.
`pid == 0`	Send the signal to all processes in the sender's process group.
`pid < 0`	Send the signal to all processes in the process group whose PGID is `abs(pid)`.
Code	

```
int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process: Run an infinite loop
        while (1) {
```

```
            printf("[Child] Running...\n");
            sleep(1);
        }
    } else {
        // Parent process: Sleep for 3 seconds then use kill() to terminate child process
        sleep(3);
        printf("[Parent] Sending SIGKILL to terminate child process (PID: %d)\n", pid);

        kill(pid, SIGKILL); // Send SIGKILL to child PID
    }

    return 0;
}
```

Output

```
[Child] Running...
[Child] Running...
[Child] Running...
[Parent] Sending SIGKILL to terminate child process (PID: 21471)
```

7.1.2 raise

raise: allows a process to send a signal to itself (equivalent to kill(getpid(), sig))

```
int raise(int signo);     <=>  int kill(getpid(), signo);
```

```
void handle_usr1(int sig) {
    write(STDOUT_FILENO, "Caught SIGUSR1 sent by this process itself!\n", 44);
}

int main() {
    // Register handler for SIGUSR1
    signal(SIGUSR1, handle_usr1);

    printf("Preparing to send SIGUSR1 to self...\n");

    raise(SIGUSR1); // Send SIGUSR1 to self

    printf("Program finished!\n");
    return 0;
```

```
}
```

Output

```
Preparing to send SIGUSR1 to self...
Caught SIGUSR1 sent by this process itself!
Program finished!
```

7.2 alarm/pause

7.2.1 alarm

Function: Schedules a SIGALRM signal to be sent after a specified number of seconds.

Default Action: If SIGALRM is not handled or ignored, the process will be forcibly terminated.

Latency: The signal is delivered on time, but the process may handle it slightly later due to CPU scheduling delays.

Alarm Clock Management Rules:

- Single Alarm Limit: Each process has only one alarm timer.
- Overwrite: If alarm() is called before a previous timer expires, the old timer is replaced by the new value. The function returns the remaining seconds of the old timer.
- Cancel Alarm: Call alarm(0) to cancel the current timer without setting a new one (it still returns the remaining seconds).

```
unsigned int alarm(unsigned int seconds);
```

Example

```
      alarm(5)
        │
        │ wait approximately 5 seconds
        ▼ SIGALRM
        │
        ▼ Signal handler
```

7.2.2 pause

Function suspends the calling process until a signal is caught.

Exit Condition: The pause() function only returns after a signal handler is triggered and finishes execution.

Return Value: -1 and sets the error code errno = EINTR (indicating that the function was interrupted by a signal).

```
int pause(void);
```

7.2.3 alram + pause

```
void handle_alarm(int sig) {
    // Safely write message using write()
    write(STDOUT_FILENO, "\n[ALARM] 3 seconds up!\n", 39);
}

int main() {
    // 1. Register signal handler for SIGALRM
    signal(SIGALRM, handle_alarm);

    printf("1. Set timer for 3 seconds...\n");
    alarm(3); // Kernel starts 3-second countdown

    printf("2. Process enters sleep state (pause) to wait for alarm...\n");

    // 3. Pause process until any Signal is received
    pause();

    printf("3. Process woke up after Signal Handler finished executing!\n");

    // Turn off alarm (if any remains) and exit
    alarm(0);
    return 0;
}
```

Output

```
1. Set timer for 3 seconds...
2. Process enters sleep state (pause) to wait for alarm...

[ALARM] 3 seconds up!
3. Process woke up after Signal Handler finished executing!
```

7.4 signal set

Function	Description
`sigemptyset(&set)`	Clears all signals from the set.
`sigfillset(&set)`	Adds all signals to the set.
`sigaddset(&set, SIGINT)`	Adds a specific signal to the set.
`sigdelset(&set, SIGINT)`	Removes a specific signal from the set.
`sigismember(&set, SIGINT)`	Checks if a signal is in the set.

Code

```
int main() {
    sigset_t my_set;

    // 1. sigemptyset: Initialize an empty set (contains no signals)
    sigemptyset(&my_set);
    printf("1. Empty sigset initialized.\n");

    // 2. sigaddset: Add SIGINT (Ctrl+C) and SIGQUIT (Ctrl+\) to the set
    sigaddset(&my_set, SIGINT);
    sigaddset(&my_set, SIGQUIT);
    printf("2. Added SIGINT and SIGQUIT to the set.\n");

    // 3. sigismember: Check if signals are in the set
    if (sigismember(&my_set, SIGINT)) {
        printf("-> Check: SIGINT IS in the set.\n");
    }

    if (!sigismember(&my_set, SIGTERM)) {
        printf("-> Check: SIGTERM IS NOT in the set.\n");
    }

    // 4. sigdelset: Remove SIGINT from the set
    sigdelset(&my_set, SIGINT);
    printf("3. Removed SIGINT from the set.\n");

    // Re-check after removal
    if (!sigismember(&my_set, SIGINT)) {
        printf("-> Re-check: SIGINT IS NOT currently in the set.\n");
    }
```

```
    // 5. sigfillset: Fill the set (add ALL signals available on the system)
    sigfillset(&my_set);
    printf("4. Called sigfillset (added all signals to the set).\n");

    // Re-check after fill
    if (sigismember(&my_set, SIGINT)) {
        printf("-> Check: SIGINT IS currently in the set.\n");
    }

    return 0;
}
```

Output

```
1. Empty sigset initialized.
2. Added SIGINT and SIGQUIT to the set.
-> Check: SIGINT IS in the set.
-> Check: SIGTERM IS NOT in the set.
3. Removed SIGINT from the set.
-> Re-check: SIGINT IS NOT currently in the set.
4. Called sigfillset (added all signals to the set).
-> Check: SIGINT IS currently in the set.
```

Note

sigfillset() to add all signals to the block mask, there are still 2 signals that can NEVER be blocked, ignored, or caught:

```
SIGKILL (Signal 9): Terminates the process immediately.


SIGSTOP (Signal 19): Suspends (stops) the process execution.
```

The kernel is designed this way so that system administrators (root) always have a way to kill or stop a frozen process, preventing a program from intentionally blocking all signals to run forever on the system

7.5 sigpromask and sigpending

sigpromark : Used to block signals in a set, preventing them from interrupting the process while executing a critical section of code. Blocked signals will enter the pending state.

Option	Description

`SIG_BLOCK`	Adds the signals in the set to the current block mask (combines with existing blocked signals).
`SIG_UNBLOCK`	Removes the signals in the set from the current block mask.
`SIG_SETMASK`	Overwrites the current block mask completely with the specified set.

siqpending: returns the set of signals that are blocked from delivery and currently pending for the calling process

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_sigint(int sig) {
    write(STDOUT_FILENO, "\n[HANDLER] Caught and handled SIGINT (Ctrl+C)!\n", 47);
}

int main() {
    sigset_t newmask, oldmask, pendingmask;

    // 1. Register Handler for SIGINT
    signal(SIGINT, handle_sigint);

    // 2. Create Signal Set containing SIGINT
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGINT);

    // 3. Use sigprocmask to BLOCK SIGINT
    // Save the old mask into oldmask to restore later
    if (sigprocmask(SIG_BLOCK, &newmask, &oldmask) < 0) {
        perror("sigprocmask error");
        exit(1);
    }

    printf("=== BLOCKING SIGINT (Ctrl+C) FOR 5 SECONDS ===\n");
    printf("Press Ctrl+C (The program will NOT be interrupted)\n");
    sleep(5);
```

```
    // 4. Use sigpending() to check signals that are currently pending
    if (sigpending(&pendingmask) < 0) {
        perror("sigpending error");
        exit(1);
    }

    // Check if SIGINT is in the pending set
    if (sigismember(&pendingmask, SIGINT)) {
        printf("\n=> Detected: SIGINT is in PENDING state (waiting to be handled)!\n");
    } else {
        printf("\n=> No SIGINT in pending list (You did not press Ctrl+C).\n");
    }

    printf("\n=== UNBLOCKING SIGINT ===\n");
    // 5. Restore the old Signal Mask (which UNBLOCKS SIGINT)
    // At this exact moment, the pending SIGINT will be flushed by the Kernel and the Handler
will execute immediately!
    if (sigprocmask(SIG_SETMASK, &oldmask, NULL) < 0) {
        perror("sigprocmask restore error");
        exit(1);
    }

    printf("Program finished safely.\n");
    return 0;
}
```

Output

```
=== BLOCKING SIGINT (Ctrl+C) FOR 5 SECONDS ===
Press Ctrl+C(The program will NOT be interrupted)
^C
=> Detected: SIGINT is in PENDING state (waiting to be handled)!

=== UNBLOCKING SIGINT ===

[HANDLER] Caught and handled SIGINT (Ctrl+C)!
Program finished safely.
```

7.6 sigaction

sigaction: system call is the modern, safe, and most flexible signal handling mechanism in Unix/Linux operating systems, completely replacing the old signal() function.

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

Parameter	Description
`signum`	The signal code to catch (for example: `SIGINT`, `SIGTERM`), except `SIGKILL` and `SIGSTOP`.
`act`	Pointer to the new action configuration for the signal.
`oldact`	Pointer to store the previous action configuration (pass `NULL` if not needed).

struct sigaction

```
struct sigaction {
    void     (*sa_handler)(int);                          // Simple handler function
    void     (*sa_sigaction)(int, siginfo_t *, void *);  // Advanced handler function (when
SA_SIGINFO is set)
    sigset_t   sa_mask;                                   // Signals blocked while the handler
is running
    int        sa_flags;                                  // Flags to control behavior
    void     (*sa_restorer)(void);                        // Reserved for internal system use
(do not use directly)
};
```

Important sa_flags Flags

Flag	Description
`SA_SIGINFO`	Enables detailed signal information through the `sa_sigaction` pointer (such as sender PID and signal information).
`SA_RESTART`	Automatically restarts interrupted system calls (such as `read()`, `write()`, and `accept()`) after the signal handler finishes.
`SA_RESETHAND`	Resets the signal behavior to `SIG_DFL` (default action) immediately after the handler runs once.

Code:

```
void general_handler(int sig) {
    if (sig == SIGINT) {
        printf("\n[Received SIGINT] You just pressed Ctrl+C!\n");
```

```
    } else if (sig == SIGTERM) {
        printf("\n[Received SIGTERM] Program requested to terminate!\n");
        exit(0); // Terminate the program
    }
}

int main() {
    struct sigaction sa;

    sa.sa_handler = general_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    // --- Register First Signal: SIGINT (Ctrl+C) ---
    sigaction(SIGINT, &sa, NULL);

    // --- Register Second Signal: SIGTERM (kill/termination command) ---
    sigaction(SIGTERM, &sa, NULL);

    printf("Program running (PID: %d)...\n", getpid());
    printf(" Test 1: Press Ctrl+C\n");
    printf(" Test 2: Open another terminal and run: kill %d\n", getpid());

    while(1) {
        sleep(1);
    }
    return 0;
}
```

Output

```
Program running (PID: 12098)...
 Test 1: Press Ctrl+C
 Test 2: Open another terminal and run: kill 12098
^C
[Received SIGINT] You just pressed Ctrl+C!

[Received SIGTERM] Program requested to terminate!
```

7.6 sigsuspend

When we want to unblock a signal and wait for it

```
sigprocmask(SIG_UNBLOCK, &mask, NULL);
pause();
```

This can cause a **race condition**: The signal may arrive between sigprocmask() and pause(). If the signal is handled before pause() starts, pause() has no signal left to wake it up.

```
sigprocmask(SIG_UNBLOCK, &mask, NULL);
                    │
                    │  ← Signal arrives here!
                    ▼
                pause();
                  │
                  └── Waits forever because the signal
                      has already been handled.
```

Solution: sigsuspend()

sigsuspend() atomically:

- Temporarily replaces the current signal mask.
- Suspends the process and waits for a signal.

```
// Variable updated by signal handler
volatile sig_atomic_t got_signal = 0;

void handler(int sig) {
    got_signal = 1;
}

int main() {
    sigset_t newmask, oldmask, waitmask;
    struct sigaction sa;

    // 1. Set up Signal Handler for SIGINT
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    // 2. Initialize masks
```

```
    sigemptyset(&newmask);
    sigaddset(&newmask, SIGINT);
    sigemptyset(&waitmask); // Empty waitmask = no signals blocked while waiting

    // 3. Block SIGINT to protect Critical Section
    sigprocmask(SIG_BLOCK, &newmask, &oldmask);

    printf("Inside critical section (SIGINT is blocked)...\n");
    sleep(2); // Simulate data processing

    // 4. Safely wait for SIGINT using sigsuspend
    while (got_signal == 0) {
        printf("Waiting for SIGINT using sigsuspend()...\n");
        sigsuspend(&waitmask); // Temporarily unblock and wait for SIGINT
    }

    // 5. Restore original mask
    sigprocmask(SIG_SETMASK, &oldmask, NULL);
    printf("Received SIGINT! Program finished safely.\n");

    return 0;
}
```

Output

```
Inside critical section (SIGINT is blocked)...
Waiting for SIGINT using sigsuspend()...
^CReceived SIGINT! Program finished safely.
```

LAB / Studio

1. Implement a chat program between two processes over a pipe/FIFO

5.2 Client/Server properties (Client/Server + FIFO)

2. Implement producer-consumer via shared memory + semaphore

producer

```
#define SHM_NAME "/posix_shm_pc"
#define SEM_MUTEX "/sem_mutex"
#define SEM_EMPTY "/sem_empty"
#define SEM_FULL  "/sem_full"

#define BUFFER_SIZE 5

typedef struct {
    int buffer[BUFFER_SIZE];
    int in;
    int out;
} SharedData;

int main() {
    // 1. Create Shared Memory
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(SharedData));
    SharedData *shared = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);

    // Initialize index values
    shared->in = 0;
    shared->out = 0;

    // 2. Create / Open Semaphores
    sem_t *mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);           // Mutual exclusion
control
    sem_t *empty = sem_open(SEM_EMPTY, O_CREAT, 0666, BUFFER_SIZE); // Count empty slots
    sem_t *full  = sem_open(SEM_FULL,  O_CREAT, 0666, 0);           // Count filled slots

    for (int i = 1; i <= 10; i++) {
        sem_wait(empty); // Decrement empty slots (wait if full)
        sem_wait(mutex); // Lock Critical Section

        // Write data into Circular Buffer
        shared->buffer[shared->in] = i * 10;
        printf("[Producer] Produced item %d at index %d\n", i * 10, shared->in);
        shared->in = (shared->in + 1) % BUFFER_SIZE;
```

```
        sem_post(mutex); // Unlock Critical Section
        sem_post(full);  // Increment filled slots

        sleep(1); // Simulate production time
    }

    // Resource cleanup
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    sem_close(mutex);
    sem_close(empty);
    sem_close(full);

    return 0;
}
```

Consumer

```
#define SHM_NAME "/posix_shm_pc"
#define SEM_MUTEX "/sem_mutex"
#define SEM_EMPTY "/sem_empty"
#define SEM_FULL  "/sem_full"

#define BUFFER_SIZE 5

typedef struct {
    int buffer[BUFFER_SIZE];
    int in;
    int out;
} SharedData;

int main() {
    // 1. Open Shared Memory created by Producer
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    SharedData *shared = mmap(NULL, sizeof(SharedData), PROT_READ | PROT_WRITE, MAP_SHARED,
shm_fd, 0);

    // 2. Open Semaphores
    sem_t *mutex = sem_open(SEM_MUTEX, 0);
```

```
    sem_t *empty = sem_open(SEM_EMPTY, 0);
    sem_t *full  = sem_open(SEM_FULL,  0);

    for (int i = 1; i <= 10; i++) {
        sem_wait(full);  // Decrement filled slots (wait if empty)
        sem_wait(mutex); // Lock Critical Section

        // Read data from Circular Buffer
        int item = shared->buffer[shared->out];
        printf("  [Consumer] Consumed item %d at index %d\n", item, shared->out);
        shared->out = (shared->out + 1) % BUFFER_SIZE;

        sem_post(mutex); // Unlock Critical Section
        sem_post(empty); // Increment empty slots

        sleep(2); // Simulate processing time
    }

    // Resource cleanup
    munmap(shared, sizeof(SharedData));
    close(shm_fd);
    sem_close(mutex);
    sem_close(empty);
    sem_close(full);

    // Unlink and permanently remove from OS
    shm_unlink(SHM_NAME);
    sem_unlink(SEM_MUTEX);
    sem_unlink(SEM_EMPTY);
    sem_unlink(SEM_FULL);

    return 0;
}
```

Output

terminal 1: ./producer

```
[Producer] Produced item 10 at index 0
[Producer] Produced item 20 at index 1
[Producer] Produced item 30 at index 2
```

```
[Producer] Produced item 40 at index 3
[Producer] Produced item 50 at index 4
[Producer] Produced item 60 at index 0
[Producer] Produced item 70 at index 1
[Producer] Produced item 80 at index 2
[Producer] Produced item 90 at index 3
[Producer] Produced item 100 at index 4
```

terminal 2: ./consumer

```
[Consumer] Consumed item 10 at index 0
[Consumer] Consumed item 20 at index 1
[Consumer] Consumed item 30 at index 2
[Consumer] Consumed item 40 at index 3
[Consumer] Consumed item 50 at index 4
[Consumer] Consumed item 60 at index 0
[Consumer] Consumed item 70 at index 1
[Consumer] Consumed item 80 at index 2
[Consumer] Consumed item 90 at index 3
[Consumer] Consumed item 100 at index 4
```

3. Register a signal handler for SIGINT/SIGALRM; use strace to observe all the IPC-related syscalls

```
// Signal handler function for SIGINT (Ctrl + C)
void handle_sigint(int sig) {
    printf("\n[Signal] Received SIGINT (%d)!\n", sig);
}

// Signal handler function for SIGALRM
void handle_sigalrm(int sig) {
    printf("[Signal] Received SIGALRM (%d) - Alarm timer expired!\n", sig);
}

int main() {
    // 1. Register signal handler for SIGINT using sigaction
    struct sigaction sa_int;
    sa_int.sa_handler = handle_sigint;
    sigemptyset(&sa_int.sa_mask);
    sa_int.sa_flags = 0;
```

```
    if (sigaction(SIGINT, &sa_int, NULL) < 0) {
        perror("Error registering SIGINT");
        return 1;
    }

    // 2. Register signal handler for SIGALRM using signal
    if (signal(SIGALRM, handle_sigalrm) == SIG_ERR) {
        perror("Error registering SIGALRM");
        return 1;
    }

    printf("Process PID: %d\n", getpid());
    printf("Signal Handlers registered. Setting alarm for 3 seconds...\n");

    // Schedule a SIGALRM signal to be delivered after 3 seconds
    alarm(3);

    printf("Waiting for signals (Press Ctrl+C to test SIGINT)... \n");

    // Wait for incoming signals
    for (int i = 0; i < 2; i++) {
        pause(); // Suspend process execution until a signal is received
    }

    printf("Program terminated.\n");
    return 0;
}
```

Output: strace -e trace=signal ./test

```
rt_sigaction(SIGINT, {sa_handler=0x565bbbb6f269, sa_mask=[], sa_flags=SA_RESTORER,
sa_restorer=0x7dab3fa45330}, NULL, 8) = 0
rt_sigaction(SIGALRM, {sa_handler=0x565bbbb6f294, sa_mask=[ALRM],
sa_flags=SA_RESTORER|SA_RESTART, sa_restorer=0x7dab3fa45330}, {sa_handler=SIG_DFL, sa_mask=[],
sa_flags=0}, 8) = 0
Process PID: 10643
Signal Handlers registered. Setting alarm for 3 seconds...
Waiting for signals (Press Ctrl+C to test SIGINT)...
pause(^C)                               = ? ERESTARTNOHAND (To be restarted if no handler)
```

```
strace: Process 10643 detached

[Signal] Received SIGINT (2)!

dat@dat-asus:~/Documents/Zhone-Training/Section4$ [Signal] Received SIGALRM (14) - Alarm timer expired!
Program terminated.
```

Explain

rt_sigaction(SIGINT, ...)

Field	Description
`sa_handler = 0x58d668ae5269`	Function pointer address of `handle_sigint` in the process's virtual memory. The signal handler is called when `SIGINT` is delivered.
`sa_mask = []`	Signal mask that is additionally blocked while `handle_sigint` is executing. An empty `[]` means no additional signals are blocked by `sa_mask`. The signal currently being handled is normally blocked automatically.
`sa_flags = SA_RESTORER`	Internal flag used by Linux/glibc to specify a signal-restoration function. It is normally not set explicitly in application code.
`sa_restorer = 0x71fdfa445330`	Address of the signal-restoration code used when returning from the signal handler. It helps the process restore its previous execution context after the handler finishes.

rt_sigaction(SIGALRM, ...)

Field	Description
`sa_handler = 0x58d668ae5294`	Function pointer address of `handle_sigalrm`.
`sa_mask = [ALRM]`	While `handle_sigalrm` is executing, `SIGALRM` is temporarily blocked. If another `SIGALRM` arrives during this time, it becomes pending instead of immediately re-entering the handler.
`sa_flags = SA_RESTORER | SA_RESTART`	`SA_RESTORER` is internal signal-restoration support used by Linux/glibc. `SA_RESTART` requests automatic restarting of certain interrupted system calls after the signal handler returns.

Revision #5
Created 2026-09-15 09:09:54 UTC by Dat Nguyen
Updated 2026-09-21 02:21:55 UTC by Dat Nguyen